Session – 17

Task-1.

Create a SQL table called Restaurant with columns: id, name, cuisine, location, and average_rating. Insert at least 5 sample rows representing popular restaurants from Zomato.

Solution:

```
CREATE TABLE Restaurant (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  cuisine VARCHAR(50),
  location VARCHAR(100),
  average_rating DECIMAL(2,1)
);
INSERT INTO Restaurant
(id, name, cuisine, location, average_rating)
VALUES
(1, 'Swagat Restaurant', 'Gujarati', 'Ahmedabad', 4.5),
(2, 'Pizza Palace', 'Italian', 'Ahmedabad', 4.2),
(3, 'Dragon House', 'Chinese', 'Surat', 4.3),
(4, 'Spice Garden', 'Indian', 'Vadodara', 4.1),
(5, 'Royal Biryani', 'Indian', 'Ahmedabad', 4.6);
```

Task-2.

Write a SQL query to generate a report showing the number of restaurants for each cuisine type from your Restaurant table, ordered by the count in descending order.

Hint: Use GROUP BY and ORDER BY.

Solution:

```
SELECT
  cuisine,
  COUNT(*) AS restaurant_count
FROM Restaurant
GROUP BY cuisine
```

```
ORDER BY restaurant_count DESC;
```

Task-3.

Add a new table called Review with columns: id, restaurant_id, user_name, rating, and review_date. Insert at least 10 sample reviews, linking them to restaurants using restaurant_id.

Solution:

```
CREATE TABLE Review (
  id INT PRIMARY KEY,
  restaurant_id INT,
  user_name VARCHAR(100),
  rating DECIMAL(2,1),
  review_date DATE,

  FOREIGN KEY (restaurant_id)
    REFERENCES Restaurant(id)
);

INSERT INTO Review
(id, restaurant_id, user_name, rating, review_date)
VALUES
(1, 1, 'Rahul', 5.0, '2026-10-01'),
(2, 1, 'Priya', 4.5, '2026-10-02'),
(3, 2, 'Amit', 4.0, '2026-10-03'),
(4, 2, 'Neha', 4.5, '2026-10-04'),
(5, 3, 'Riya', 4.5, '2026-10-05'),
(6, 3, 'Karan', 4.0, '2026-10-06'),
(7, 4, 'Meera', 3.5, '2026-10-07'),
(8, 4, 'Arjun', 4.0, '2026-10-08'),
(9, 5, 'Vivek', 5.0, '2026-10-09'),
(10, 5, 'Anjali', 4.5, '2026-10-10');
```

Task-4.

Write a SQL query using a JOIN to display each restaurant's name, cuisine, and its average review rating (from the Review table), ordered by highest average rating first.

Hint: Use JOIN and GROUP BY with aggregate functions.

Solution:

```
SELECT
  r.name,
  r.cuisine,
  AVG(rv.rating) AS average_review_rating
FROM Restaurant r
JOIN Review rv
  ON r.id = rv.restaurant_id
GROUP BY
  r.id,
  r.name,
  r.cuisine
ORDER BY average_review_rating DESC;
```

Task-5.

Use a window function to rank restaurants by their average review rating within each cuisine type, showing the restaurant name, cuisine, average rating, and rank.

Hint: Use the RANK() or DENSE_RANK() window function partitioned by cuisine.

Solution:

```
WITH RestaurantRatings AS (
  SELECT
    r.id,
    r.name,
    r.cuisine,
    AVG(rv.rating) AS average_rating
  FROM Restaurant r
```

```
    JOIN Review rv
        ON r.id = rv.restaurant_id
    GROUP BY
        r.id,
        r.name,
        r.cuisine
)

SELECT
    name,
    cuisine,
    average_rating,
    RANK() OVER (
        PARTITION BY cuisine
        ORDER BY average_rating DESC
    ) AS cuisine_rank
FROM RestaurantRatings
ORDER BY cuisine, cuisine_rank;
```